

Kotlin Higher-Order Functions

50 Questions with Complete Code Examples

Master functional programming in Kotlin: map, filter, reduce, fold, flatMap, and dozens more higher-order functions. Learn to write expressive, concise code using lambdas and function references.

EASY LEVEL (1-20)

Q1: What is a higher-order function?

A function that takes functions as parameters or returns a function.

```
// Takes function as parameter fun operate(x: Int, y: Int, operation: (Int, Int) -> Int): Int { return operation(x, y) } // Usage val sum = operate(5, 3) { a, b -> a + b } val product = operate(5, 3) { a, b -> a * b } println(sum) // 8 println(product) // 15
```

■ *Foundation of functional programming in Kotlin*

Q2: How do you define a function that takes a lambda?

Use function type syntax: (ParameterTypes) -> ReturnType

```
fun processString(text: String, transformer: (String) -> String): String { return transformer(text) } // Usage with different lambdas val upper = processString("hello") { it.uppercase() } val reversed = processString("hello") { it.reversed() } val doubled = processString("hello") { it + it } println(upper) // HELLO println(reversed) // olleh println(doubled) // hellohello
```

■ *Function types are first-class citizens*

Q3: What is the 'it' keyword in lambdas?

Implicit name for single parameter in lambda expressions.

```
val numbers = listOf(1, 2, 3, 4, 5) // Using 'it' val doubled = numbers.map { it * 2 } val evens = numbers.filter { it % 2 == 0 } // Explicit parameter name val tripled = numbers.map { num -> num * 3 } println(doubled) // [2, 4, 6, 8, 10] println(evens) // [2, 4] println(tripled) // [3, 6, 9, 12, 15]
```

■ *'it' makes code concise for single-parameter lambdas*

Q4: How to use map() function?

Transforms each element in a collection using a lambda.

```
val numbers = listOf(1, 2, 3, 4, 5) // Transform to squares val squares = numbers.map { it * it } // Transform to strings val strings = numbers.map { "Number: $it" } data class User(val name: String, val age: Int) val users = listOf(User("Alice", 25), User("Bob", 30)) // Extract names val names = users.map { it.name } println(squares) // [1, 4, 9, 16, 25] println(strings) // [Number: 1, Number: 2, ...] println(names) // [Alice, Bob]
```

■ *Most common collection transformation function*

Q5: How does filter() work?

Returns elements that match the predicate (condition).

```
val numbers = listOf(1, 2, 3, 4, 5, 6, 7, 8, 9, 10) // Filter even numbers val evens = numbers.filter { it % 2 == 0 } // Filter numbers greater than 5 val greaterThan5 = numbers.filter { it > 5 } // Filter and map combined val evenSquares = numbers .filter { it % 2 == 0 } .map { it * it } data class Product(val name: String, val price: Double, val inStock: Boolean) val products = listOf( Product("Laptop", 999.99, true), Product("Phone", 599.99, false), Product("Tablet", 399.99, true) ) val available = products.filter { it.inStock } val affordable = products.filter { it.price < 500 } println(evens) // [2, 4, 6, 8, 10] println(evenSquares) // [4, 16, 36, 64, 100]
```

■ *Essential for filtering collections by criteria*

Q6: What does forEach() do?

Performs an action on each element (side effects).

```
val names = listOf("Alice", "Bob", "Charlie") // Print each name names.forEach { println(it) } // With index names.forEachIndexed { index, name -> println("$index: $name") } // Modify external state var sum = 0 listOf(1, 2, 3, 4, 5).forEach { sum += it } println("Sum: $sum") // Sum: 15 // Better: use reduce for this val total = listOf(1, 2, 3, 4, 5).reduce { acc, i -> acc + i } println("Total: $total") // Total: 15
```

■ *Use for side effects; prefer map/filter for transformations*

Q7: Explain reduce() function

Accumulates values by applying operation from left to right.

```
val numbers = listOf(1, 2, 3, 4, 5) // Sum all numbers val sum = numbers.reduce { acc, num -> acc + num } // Find maximum val max = numbers.reduce { acc, num -> if (num > acc) num else acc } // Concatenate strings val words = listOf("Kotlin", "is", "awesome") val sentence = words.reduce { acc, word -> "$acc $word" } println(sum) // 15 println(max) // 5 println(sentence) // Kotlin is awesome // reduceOrNull for empty lists val empty = emptyList<Int>() val result = empty.reduceOrNull { acc, i -> acc + i } println(result) // null
```

■ *Reduces collection to single value; throws on empty list*

Q8: What's the difference between reduce() and fold()?

fold() takes initial value; reduce() uses first element.

```
val numbers = listOf(1, 2, 3, 4, 5) // reduce - uses first element as initial val reduceSum = numbers.reduce { acc, i -> acc + i } //  
fold - provides initial value val foldSum = numbers.fold(0) { acc, i -> acc + i } val foldSum100 = numbers.fold(100) { acc, i -> acc + i  
} println(reduceSum) // 15 println(foldSum) // 15 println(foldSum100) // 115 // fold works on empty lists val empty = emptyList<Int>()  
val result = empty.fold(0) { acc, i -> acc + i } println(result) // 0 // Useful for type conversion val concatenated = numbers.fold("")  
{ acc, i -> "$acc$i" } println(concatenated) // 12345
```

■ *fold is safer and more flexible than reduce*

Q9: How to use find() and firstOrNull()?

find() returns first matching element or null.

```
val numbers = listOf(1, 2, 3, 4, 5, 6, 7, 8, 9, 10) // Find first even number val firstEven = numbers.find { it % 2 == 0 } //  
firstOrNull is alias for find val firstGreaterThan5 = numbers.firstOrNull { it > 5 } // last matching element val lastEven =  
numbers.lastOrNull { it % 2 == 0 } data class User(val name: String, val age: Int, val active: Boolean) val users = listOf(  
User("Alice", 25, true), User("Bob", 30, false), User("Charlie", 35, true) ) val firstActive = users.find { it.active } val over30 =  
users.find { it.age > 30 } println(firstEven) // 2 println(lastEven) // 10 println(firstActive) // User(Alice, 25, true)  
println(over30) // User(Charlie, 35, true)
```

■ *Returns null if no match found; stops at first match*

Q10: What does any(), all(), none() do?

Check if any/all/none elements match a predicate.

```
val numbers = listOf(1, 2, 3, 4, 5) // any - at least one matches val hasEven = numbers.any { it % 2 == 0 } // all - all elements match  
val allPositive = numbers.all { it > 0 } val allEven = numbers.all { it % 2 == 0 } // none - no elements match val noneNegative =  
numbers.none { it < 0 } println(hasEven) // true println(allPositive) // true println(allEven) // false println(noneNegative) // true  
// With custom objects data class User(val name: String, val age: Int) val users = listOf(User("Alice", 17), User("Bob", 25)) val  
hasAdults = users.any { it.age >= 18 } val allAdults = users.all { it.age >= 18 } println(hasAdults) // true println(allAdults) //  
false
```

■ *Short-circuits: stops as soon as result is determined*

Q11: How to use groupBy()?

Groups elements by a key function into a Map.

```
data class Student(val name: String, val grade: Char, val score: Int) val students = listOf( Student("Alice", 'A', 95), Student("Bob",  
'B', 85), Student("Charlie", 'A', 92), Student("David", 'C', 75), Student("Eve", 'B', 88) ) // Group by grade val byGrade =  
students.groupBy { it.grade } // Group by score range val byRange = students.groupBy { when { it.score >= 90 -> "Excellent" it.score >=  
80 -> "Good" else -> "Average" } } println(byGrade) // {A=[Student(Alice, A, 95), Student(Charlie, A, 92)], // B=[Student(Bob, B, 85),  
Student(Eve, B, 88)], // C=[Student(David, C, 75)]} // Count by grade val countByGrade = byGrade.mapValues { it.value.size }  
println(countByGrade) // {A=2, B=2, C=1}
```

■ *Creates Map> from collection*

Q12: Explain partition() function

Splits collection into two lists based on predicate.

```
val numbers = listOf(1, 2, 3, 4, 5, 6, 7, 8, 9, 10) // Partition into evens and odds val (evens, odds) = numbers.partition { it % 2 == 0  
} println(evens) // [2, 4, 6, 8, 10] println(odds) // [1, 3, 5, 7, 9] data class Task(val name: String, val completed: Boolean, val  
priority: Int) val tasks = listOf( Task("Code review", true, 1), Task("Write tests", false, 2), Task("Fix bugs", false, 1),  
Task("Deploy", true, 3) ) // Partition by completion status val (completed, pending) = tasks.partition { it.completed } // Partition by  
priority val (highPriority, lowPriority) = tasks.partition { it.priority <= 2 } println("Completed: ${completed.map { it.name }}")  
println("Pending: ${pending.map { it.name }}")
```

■ *Returns Pair - true matches, false matches*

Q13: How does take() and drop() work?

take() gets first n elements; drop() skips first n.

```
val numbers = listOf(1, 2, 3, 4, 5, 6, 7, 8, 9, 10) // Take first 5 val first5 = numbers.take(5) // Drop first 5 val afterFirst5 =  
numbers.drop(5) // takeWhile - take until condition false val lessThan6 = numbers.takeWhile { it < 6 } // dropWhile - drop until  
condition false val from6 = numbers.dropWhile { it < 6 } // takeLast and dropLast val last3 = numbers.takeLast(3) val exceptLast3 =  
numbers.dropLast(3) println(first5) // [1, 2, 3, 4, 5] println(afterFirst5) // [6, 7, 8, 9, 10] println(lessThan6) // [1, 2, 3, 4, 5]  
println(from6) // [6, 7, 8, 9, 10] println(last3) // [8, 9, 10] println(exceptLast3) // [1, 2, 3, 4, 5, 6, 7]
```

■ *Useful for pagination and slicing collections*

Q14: What is flatMap()?

Maps each element to a collection and flattens the result.

```
data class Team(val name: String, val members: List<String>) val teams = listOf( Team("Backend", listOf("Alice", "Bob")),
Team("Frontend", listOf("Charlie", "David", "Eve")), Team("DevOps", listOf("Frank")) ) // Get all members from all teams val allMembers
= teams.flatMap { it.members } println(allMembers) // [Alice, Bob, Charlie, David, Eve, Frank] // flatMap vs map val mapped = teams.map
{ it.members } println(mapped) // [[Alice, Bob], [Charlie, David, Eve], [Frank]] // String example val words = listOf("Hello", "World")
val chars = words.flatMap { it.toList() } println(chars) // [H, e, l, l, o, w, o, r, l, d] // With transformation val upperMembers =
teams.flatMap { team -> team.members.map { it.uppercase() } } println(upperMembers) // [ALICE, BOB, CHARLIE, DAVID, EVE, FRANK]
```

■ Combines map and flatten operations

Q15: How to use associateBy()?

Creates Map using lambda to generate keys.

```
data class User(val id: Int, val name: String, val email: String) val users = listOf( User(1, "Alice", "alice@example.com"), User(2,
"Bob", "bob@example.com"), User(3, "Charlie", "charlie@example.com") ) // Map by ID val byId = users.associateBy { it.id }
println(byId[1]) // User(1, Alice, alice@example.com) // Map by email val byEmail = users.associateBy { it.email }
println(byEmail["bob@example.com"]) // User(2, Bob, ...) // With value transformer val idToName = users.associateBy({ it.id }, {
it.name }) println(idToName) // {1=Alice, 2=Bob, 3=Charlie} // associate - provide both key and value val emailToName = users.associate
{ it.email to it.name } println(emailToName) // {alice@example.com=Alice, bob@example.com=Bob, ...}
```

■ Creates lookup maps for fast access by key

Q16: What does sortBy() do?

Returns sorted list based on selector function.

```
data class Person(val name: String, val age: Int, val score: Double) val people = listOf( Person("Alice", 25, 95.5), Person("Bob", 30,
87.3), Person("Charlie", 25, 92.1), Person("David", 28, 95.5) ) // Sort by age val byAge = people.sortBy { it.age } // Sort by name
val byName = people.sortBy { it.name } // Sort by score (descending) val byScoreDesc = people.sortByDescending { it.score } // Sort
by multiple criteria val sorted = people.sortedWith( compareBy({ it.age }, { it.name }) ) println(byAge.map { "${it.name}: ${it.age}"
}) // [Alice: 25, Charlie: 25, David: 28, Bob: 30] println(byScoreDesc.map { "${it.name}: ${it.score}" }) // [Alice: 95.5, David: 95.5,
Charlie: 92.1, Bob: 87.3]
```

■ Non-destructive; returns new sorted list

Q17: How to use distinctBy()?

Returns list with distinct elements based on selector.

```
data class Product(val id: Int, val name: String, val category: String) val products = listOf( Product(1, "Laptop", "Electronics"),
Product(2, "Phone", "Electronics"), Product(3, "Desk", "Furniture"), Product(4, "Tablet", "Electronics"), Product(5, "Chair",
"Furniture") ) // Distinct categories val categories = products.distinctBy { it.category } println(categories.map { it.category }) //
[Electronics, Furniture] // Simple distinct val numbers = listOf(1, 2, 2, 3, 3, 3, 4, 5, 5) val unique = numbers.distinct()
println(unique) // [1, 2, 3, 4, 5] data class User(val name: String, val email: String) val users = listOf( User("Alice",
"alice@ex.com"), User("Bob", "bob@ex.com"), User("Alice", "alice2@ex.com") ) val uniqueNames = users.distinctBy { it.name }
println(uniqueNames.map { it.name }) // [Alice, Bob]
```

■ Keeps first occurrence of each distinct element

Q18: Explain maxBy() and minBy()

Returns element with max/min value of selector.

```
data class Player(val name: String, val score: Int, val level: Int) val players = listOf( Player("Alice", 1500, 10), Player("Bob",
2000, 8), Player("Charlie", 1800, 12), Player("David", 2200, 15) ) // Player with highest score val topScorer = players.maxByOrNull {
it.score } // Player with highest level val topLevel = players.maxByOrNull { it.level } // Player with lowest score val lowestScorer =
players.minByOrNull { it.score } println(topScorer) // Player(David, 2200, 15) println(topLevel) // Player(David, 2200, 15)
println(lowestScorer) // Player(Alice, 1500, 10) // With numbers val numbers = listOf(5, 2, 8, 1, 9, 3) println(numbers.maxOrNull()) //
9 println(numbers.minOrNull()) // 1 // sumOf, maxOf, minOf val totalScore = players.sumOf { it.score } val highestScore = players.maxOf
{ it.score } println(totalScore) // 7500 println(highestScore) // 2200
```

■ Returns null for empty collections (use OrNull variants)

Q19: What is chunked()?

Splits collection into chunks of specified size.

```
val numbers = (1..10).toList() // Chunk into groups of 3 val chunks = numbers.chunked(3) println(chunks) // [[1, 2, 3], [4, 5, 6], [7,
8, 9], [10]] // Chunk with transformation val sums = numbers.chunked(3) { it.sum() } println(sums) // [6, 15, 24, 10] // Processing in
batches data class Task(val id: Int, val name: String) val tasks = (1..100).map { Task(it, "Task $it") } // Process in batches of 10
tasks.chunked(10).forEach { batch -> println("Processing batch of ${batch.size} tasks") } // Process batch... } // Pagination val page1 =
tasks.chunked(20)[0] // First 20 items val page2 = tasks.chunked(20)[1] // Next 20 items
```

■ Last chunk may be smaller if not evenly divisible

Q20: How does windowed() work?

Creates sliding windows of specified size over collection.

```
val numbers = listOf(1, 2, 3, 4, 5, 6, 7, 8, 9, 10) // Windows of size 3 val windows = numbers.windowed(3) println(windows) //
[[1,2,3], [2,3,4], [3,4,5], [4,5,6], [5,6,7], [6,7,8], [7,8,9], [8,9,10]] // With step (skip elements) val stepped =
numbers.windowed(size = 3, step = 2) println(stepped) // [[1,2,3], [3,4,5], [5,6,7], [7,8,9], [9,10]] // With transformation val
averages = numbers.windowed(3) { it.average() } println(averages) // [2.0, 3.0, 4.0, 5.0, 6.0, 7.0, 8.0, 9.0] // Calculate moving sum
val prices = listOf(10.0, 12.0, 11.0, 15.0, 13.0, 14.0) val movingAvg = prices.windowed(3) { it.average() } println(movingAvg) //
[11.0, 12.67, 13.0, 14.0] // partialWindows = true includes smaller windows at end val partial = numbers.windowed(3, partialWindows =
true)
```

■ *Useful for moving averages, pattern detection*